COMPLETE CLASSROOM NOTES

RAG CODING WITH PYTHON

15 coding questions with simple snippets and explanations

PDF | DOCX | TXT -> Chunks -> Embeddings

-> ChromaDB -> Top-3 Search -> LLM Answer + Sources

Beginner-friendly | Interview-ready | FastAPI mini-project included

Learning objectives

By the end of these notes, a learner can build a small Retrieval-Augmented Generation (RAG) application that uploads documents, stores their meaning as vectors, retrieves relevant text, and asks an LLM to answer only from that text.

What is RAG?

RAG means **Retrieval-Augmented Generation**. Before the LLM writes an answer, the application searches a private document collection and supplies the most relevant passages as context. This improves grounding and lets the response show its source.

Stage	Simple meaning
Load	Read text from PDF, DOCX or TXT
Chunk	Break long text into small passages
Embed	Convert each passage into a numeric vector
Store	Save vectors and metadata in ChromaDB
Retrieve	Find the top three passages related to a question
Generate	Ask an LLM to answer from the retrieved passages
Cite	Return filename and page number

Core flow

Document -> extracted pages -> chunks -> embeddings -> vector database. A question follows the same embedding step, then similarity search returns context, and the LLM produces the final grounded answer.

Project setup

Recommended project structure

```
rag-project/
|-- main.py
|-- uploads/
|-- chroma_data/
`-- .env
```

Installation commands

```
py -m venv venv
venv\Scripts\activate
py -m pip install fastapi uvicorn python-multipart openai chromadb pymupdf python-docx
python-dotenv
```

Create a .env file

```
OPENAI_API_KEY=place_your_own_api_key_here
```

Never paste a real API key into source code or commit it to Git.

Common setup used by the snippets

```
from openai import OpenAI
```

```
import chromadb

client = OpenAI()

chroma_client = chromadb.PersistentClient(path='./chroma_data')
collection = chroma_client.get_or_create_collection(name='documents')
```

1. Load text from a PDF document

Purpose

PDF files store content page by page. PyMuPDF opens the file, reads each page, and preserves the page number as metadata.

Simple Python code

```
import fitz

def load_pdf(file_path):
    pages = []
    pdf = fitz.open(file_path)

    for index, page in enumerate(pdf):
        pages.append({
            'text': page.get_text(),
            'page': index + 1
        })

    pdf.close()
    return pages

pages = load_pdf('sample.pdf')
print(pages[0]['text'])
```

Explanation

The function returns a list of dictionaries. Every dictionary contains page text and its human-readable page number. Keeping page numbers now makes source display easy later.

Student checkpoint

Run the snippet, print its output, and explain what data enters and leaves this step.

2. Split text into smaller chunks

Purpose

An embedding model should receive focused passages instead of an entire long book. Overlap preserves sentences that cross a chunk boundary.

Simple Python code

```
def split_text(text, chunk_size=500, overlap=50):
    chunks = []
    start = 0

    while start < len(text):
        end = start + chunk_size
        chunks.append(text[start:end])
        start = end - overlap

    return chunks

chunks = split_text('A very long document text...')
print(chunks)
```

Explanation

Here, each chunk has at most 500 characters. The next chunk starts 50 characters before the previous one ended. In production, token-aware or sentence-aware splitting is usually better.

Student checkpoint

Run the snippet, print its output, and explain what data enters and leaves this step.

3. Generate embeddings for each chunk

Purpose

An embedding is a numeric list that represents semantic meaning. Similar passages normally have nearby vectors.

Simple Python code

```
def create_embedding(text):
    response = client.embeddings.create(
        model='text-embedding-3-small',
        input=text
    )
    return response.data[0].embedding

chunks = ['Python is simple.', 'FastAPI builds APIs.']
embeddings = [create_embedding(chunk) for chunk in chunks]
print(len(embeddings))
```

Explanation

The function sends one text string to the embedding model and returns its vector. Use the same model for document chunks and questions.

Student checkpoint

Run the snippet, print its output, and explain what data enters and leaves this step.

4. Store embeddings in ChromaDB

Purpose

ChromaDB stores the vector together with original text and metadata. IDs must be unique.

Simple Python code

```
chunks = ['Python is simple.', 'FastAPI builds APIs.']
embeddings = [create_embedding(chunk) for chunk in chunks]

collection.add(
    ids=['chunk-1', 'chunk-2'],
    documents=chunks,
    embeddings=embeddings,
    metadatas=[
        {'source': 'python.pdf', 'page': 1, 'category': 'python'},
        {'source': 'python.pdf', 'page': 2, 'category': 'api'}
    ]
)

print('Stored successfully')
```

Explanation

documents keeps readable passages; embeddings enables semantic search; metadatas keeps filename, page and category; ids identifies every stored chunk.

Student checkpoint

Run the snippet, print its output, and explain what data enters and leaves this step.

5. Convert a user's question into an embedding

Purpose

The question must be converted into the same vector space as the stored document chunks.

Simple Python code

```
question = 'What is FastAPI?'
question_embedding = create_embedding(question)
print(question_embedding[:5])
```

Explanation

The first five numbers are printed only for demonstration. The complete vector is sent to ChromaDB for comparison.

Student checkpoint

Run the snippet, print its output, and explain what data enters and leaves this step.

6. Retrieve the top three similar chunks

Purpose

Similarity search ranks stored vectors according to their closeness to the question vector.

Simple Python code

```
results = collection.query(
    query_embeddings=[question_embedding],
    n_results=3
)

top_chunks = results['documents'][0]

for chunk in top_chunks:
    print(chunk)
```

Explanation

n_results=3 means top-k is three. The first [0] is required because ChromaDB can accept multiple questions in one query and returns one result list per question.

Student checkpoint

Run the snippet, print its output, and explain what data enters and leaves this step.

7. Pass retrieved context and question to an LLM

Purpose

Join the retrieved chunks into one context block and ask the model to answer from it.

Simple Python code

```
question = 'What is FastAPI?'
context = '\n\n'.join(top_chunks)

prompt = f'''
Use the context to answer the question.

Context:
{context}

Question:
{question}
'''

response = client.responses.create(
    model='gpt-5.5',
    input=prompt
)

answer = response.output_text
print(answer)
```

Explanation

Retrieval and generation are separate steps: ChromaDB selects evidence; the LLM turns that evidence into a natural-language answer. Use a text model available to your OpenAI project.

Student checkpoint

Run the snippet, print its output, and explain what data enters and leaves this step.

8. Display answer with source and page number

Purpose

Metadata from the retrieved chunks can be returned beside the answer.

Simple Python code

```
print('Answer:', answer)
print('Sources:')

seen = set()
for item in results['metadatas'][0]:
    source = f"{item['source']} - Page {item['page']}"
    if source not in seen:
        print(source)
        seen.add(source)
```

Explanation

The set prevents the same filename and page from appearing repeatedly when two retrieved chunks came from the same page.

Student checkpoint

Run the snippet, print its output, and explain what data enters and leaves this step.

9. Add conversation memory

Purpose

A simple list can remember earlier user and assistant messages while the server process is running.

Simple Python code

```
conversation_memory = []

def ask_with_memory(question, context):
    conversation_memory.append({
        'role': 'user', 'content': question
    })

    history = '\n'.join(
        f"{m['role']}: {m['content']}"
        for m in conversation_memory
    )

    response = client.responses.create(
        model='gpt-5.5',
        input=f'''History:
{history}

Context:
{context}

Question:
{question}
Answer only from context.'''
    )

    answer = response.output_text
    conversation_memory.append({
        'role': 'assistant', 'content': answer
    })
    return answer
```

Explanation

This is teaching-level memory. It disappears when the application restarts and is shared by all users. A real application should store memory by session ID in a database.

Student checkpoint

Run the snippet, print its output, and explain what data enters and leaves this step.

10. Build POST /upload

Purpose

The endpoint saves an uploaded file, extracts its pages, creates chunks and embeddings, and stores them in ChromaDB.

Simple Python code

```
from fastapi import FastAPI, UploadFile, File
import os, uuid

app = FastAPI()
os.makedirs('uploads', exist_ok=True)

@app.post('/upload')
async def upload(file: UploadFile = File(...)):
    path = os.path.join('uploads', file.filename)

    with open(path, 'wb') as output:
        output.write(await file.read())

    pages = load_document(path)
    count = 0

    for page in pages:
        for chunk in split_text(page['text']):
            collection.add(
                ids=[str(uuid.uuid4())],
                documents=[chunk],
                embeddings=[create_embedding(chunk)],
                metadatas=[{
                    'source': file.filename,
                    'page': page['page']
                }]
            )
            count += 1

    return {'message': 'Uploaded', 'chunks': count}
```

Explanation

UploadFile receives multipart file data. UUID creates a unique ID for every chunk. The helper load_document is defined in Question 12.

Student checkpoint

Run the snippet, print its output, and explain what data enters and leaves this step.

11. Build POST /ask

Purpose

The endpoint receives JSON, retrieves the best chunks, prompts the LLM and returns answer plus sources.

Simple Python code

```
from pydantic import BaseModel

class QuestionRequest(BaseModel):
    question: str

@app.post('/ask')
def ask(request: QuestionRequest):
    vector = create_embedding(request.question)
    results = collection.query(
        query_embeddings=[vector], n_results=3
    )

    context = '\n\n'.join(results['documents'][0])
    prompt = f'''Answer only from this context.

{context}

Question: {request.question}
If missing, say: Answer not found in the uploaded documents.'''

    response = client.responses.create(
        model='gpt-5.5', input=prompt
    )

    return {
        'answer': response.output_text,
        'sources': results['metadatas'][0]
    }
```

Explanation

A POST body such as {"question": "What is RAG?"} is validated by Pydantic. The response contains evidence metadata for traceability.

Student checkpoint

Run the snippet, print its output, and explain what data enters and leaves this step.

12. Support PDF, DOCX and TXT

Purpose

Select the correct parser using the filename extension and return the same page structure for every format.

Simple Python code

```
import fitz
from docx import Document

def load_document(file_path):
    ext = file_path.lower().split('.')[-1]

    if ext == 'pdf':
        return load_pdf(file_path)

    if ext == 'docx':
        doc = Document(file_path)
        text = '\n'.join(p.text for p in doc.paragraphs)
        return [{'text': text, 'page': 1}]

    if ext == 'txt':
        with open(file_path, encoding='utf-8') as file:
            return [{'text': file.read(), 'page': 1}]

    raise ValueError('Only PDF, DOCX and TXT are supported')
```

Explanation

PDF has natural pages. A basic DOCX or TXT loader treats the entire file as page 1. More advanced DOCX source tracking can use headings or paragraph numbers.

Student checkpoint

Run the snippet, print its output, and explain what data enters and leaves this step.

13. Prevent answers outside retrieved context

Purpose

Use a strict grounding instruction and an exact fallback sentence.

Simple Python code

```
prompt = f'''
You are a document question-answering assistant.

Rules:
1. Use only the supplied context.
2. Do not use outside knowledge.
3. Do not guess.
4. If the answer is missing, respond exactly:
   Answer not found in the uploaded documents.

Context:
{context}

Question:
{question}
'''
```

Explanation

Prompting reduces unsupported answers but cannot mathematically guarantee zero hallucination. Good chunking, retrieval thresholds, evaluations and source display provide additional protection.

Student checkpoint

Run the snippet, print its output, and explain what data enters and leaves this step.

14. Delete a document and its embeddings

Purpose

Delete all ChromaDB records whose source metadata matches the filename, then remove the uploaded file.

Simple Python code

```
@app.delete('/documents/{filename}')
def delete_document(filename: str):
    collection.delete(where={'source': filename})

    path = os.path.join('uploads', filename)
    if os.path.exists(path):
        os.remove(path)

    return {
        'message': 'Document and embeddings deleted',
        'filename': filename
    }
```

Explanation

The metadata filter deletes every chunk belonging to that document. In a production application, prefer an internal document ID to avoid ambiguity between files with the same name.

Student checkpoint

Run the snippet, print its output, and explain what data enters and leaves this step.

15. Filter retrieval by filename or category

Purpose

Metadata filtering restricts search to a selected document or category before vector ranking.

Simple Python code

```
# Search only inside one file
results = collection.query(
    query_embeddings=[question_embedding],
    n_results=3,
    where={'source': 'python.pdf'}
)

# Search only inside one category
results = collection.query(
    query_embeddings=[question_embedding],
    n_results=3,
    where={'category': 'api'}
)
```

Explanation

Filtering is useful when the user selects a document, department, course or category. Store that value in metadata during upload so it is available during search.

Student checkpoint

Run the snippet, print its output, and explain what data enters and leaves this step.

Complete single-file mini-project

The following main.py combines the core snippets into one beginner-friendly RAG API. It intentionally stays simple for classroom demonstration.

```
import os
import uuid
import fitz
import chromadb
from docx import Document
from openai import OpenAI
from fastapi import FastAPI, UploadFile, File
from pydantic import BaseModel

app = FastAPI(title="Simple RAG Application")
client = OpenAI()
os.makedirs("uploads", exist_ok=True)

db = chromadb.PersistentClient(path="./chroma_data")
collection = db.get_or_create_collection(name="documents")

class QuestionRequest(BaseModel):
    question: str

def load_pdf(path):
    pages = []
    pdf = fitz.open(path)
    for index, page in enumerate(pdf):
        pages.append({"text": page.get_text(), "page": index + 1})
    pdf.close()
    return pages

def load_document(path):
    ext = path.lower().split(".")[-1]
    if ext == "pdf":
        return load_pdf(path)
    if ext == "docx":
        doc = Document(path)
        text = "\n".join(p.text for p in doc.paragraphs)
        return [{"text": text, "page": 1}]
    if ext == "txt":
        with open(path, encoding="utf-8") as file:
            return [{"text": file.read(), "page": 1}]
    raise ValueError("Only PDF, DOCX and TXT are supported")

def split_text(text, chunk_size=500, overlap=50):
    chunks, start = [], 0
    while start < len(text):
        chunks.append(text[start:start + chunk_size])
        start += chunk_size - overlap
    return chunks

def create_embedding(text):
    result = client.embeddings.create(
        model="text-embedding-3-small", input=text
    )
    return result.data[0].embedding

@app.post("/upload")
async def upload(file: UploadFile = File(...)):
    path = os.path.join("uploads", file.filename)
    with open(path, "wb") as output:
        output.write(await file.read())

    count = 0
    for page in load_document(path):
        for chunk in split_text(page["text"]):
            if not chunk.strip():
                continue
            collection.add(
                ids=[str(uuid.uuid4())],
                documents=[chunk],
                embeddings=[create_embedding(chunk)],
                metadatas=[{"source": file.filename, "page": page["page"]}]
            )
            count += 1
```

```
    return {"message": "Uploaded successfully", "chunks": count}

@app.post("/ask")
def ask(request: QuestionRequest):
    results = collection.query(
        query_embeddings=[create_embedding(request.question)],
        n_results=3
    )
    context = "\n\n".join(results["documents"][0])
    prompt = f"""Use only the context below.
If the answer is missing, say: Answer not found in the uploaded documents.

Context:
{context}

Question:
{request.question}"""
    response = client.responses.create(model="gpt-5.5", input=prompt)
    return {"answer": response.output_text,
            "sources": results["metadatas"][0]}

@app.delete("/documents/{filename}")
def delete_document(filename: str):
    collection.delete(where={"source": filename})
    path = os.path.join("uploads", filename)
    if os.path.exists(path):
        os.remove(path)
    return {"message": "Deleted successfully", "filename": filename}
```

Run and test the application

1. Start the server

```
uvicorn main:app --reload
```

2. Open Swagger UI

```
http://127.0.0.1:8000/docs
```

3. Test in order

- POST /upload: choose a PDF, DOCX or TXT file.
- POST /ask: send a JSON question such as {"question": "What is RAG?"}.
- Read the answer and verify its filename and page metadata.
- DELETE /documents/{filename}: remove the file and all its vectors.

Common errors

Problem	Likely solution
401 invalid_api_key	Set a valid OPENAI_API_KEY environment variable.
pip not recognized	Use py -m pip install ... on Windows.
Empty PDF text	The PDF may be scanned; OCR is required.
No results	Upload a document first and confirm chunks were stored.
Model not found	Choose a text model available in your OpenAI project.
Repeated sources	Deduplicate filename-page pairs using a set.

Interview and practice questions

Short interview questions

- What is the difference between an embedding model and an LLM?
- Why do we split documents into chunks?
- What is chunk overlap and why is it useful?
- What does top-k mean in similarity search?
- Why must questions and chunks use the same embedding model?
- What metadata should be stored with every chunk?
- How does RAG reduce hallucination?
- Why is prompt-only grounding not a complete guarantee?
- What is the difference between ChromaDB and FAISS?
- How would you isolate conversation memory for multiple users?

Hands-on assignments

- Add a category field to POST /upload and save it in metadata.
- Allow POST /ask to optionally search within one filename.
- Return unique sources only.
- Add a minimum similarity threshold before calling the LLM.
- Store conversation memory using a session_id.
- Add OCR support for scanned PDF pages.

Quick revision

RAG = Retrieve relevant chunks + Augment the prompt + Generate a grounded answer. The vector database does not write the final answer; it finds evidence. The LLM does not search ChromaDB automatically; application code retrieves and supplies the context.

References

OpenAI Vector Embeddings: https://developers.openai.com/api/docs/guides/embeddings

OpenAI Python API: https://developers.openai.com/api/reference/python

ChromaDB documentation: https://docs.trychroma.com/

FastAPI documentation: https://fastapi.tiangolo.com/